7 로봇 비전 · 인식

Robot Perception

이 단원을 마치면

카메라가 본 픽셀을 로봇이 갈 mm 로 바꾸고, 색으로 물체를 찾고, 보면서 움직이는 목표를 따라간다

먼저 배울 것 5 로봇 제어공학

24차시 **7-1 카메라와 좌표**
카메라가 본 "사진 속 위치" 가 있다. 이것을 로봇이 갈 "책상 위 위치" 로 어떻게 바꿀까?

25차시 **7-2 색으로 찾기**
카메라는 사진 속 빨간 블록을 어떻게 찾을까? 방이 어두워지면 왜 놓칠까?

26차시 **7-3 시각 서보**
블록이 움직인다. 카메라로 보면서 따라가려면 어떻게 해야 할까?

이 교재 쓰는 법: 한 주제가 한 차시(50분)

도입 학습 목표를 읽고, **생각 열기** 질문에 내 생각을 적어 본 뒤 그림을 봐요.

개념 정의 · 원리 · 자세히를 읽어요. 처음 보는 기호는 위에 작은 이름이 있고, 주제마다 **기호 표**가 있어요. 대학 과정은 수식까지.

탐구 활동 1 → 2 → 3 시뮬레이터 [교육] 탭에서 같은 동작을 한 가지만 바꿔 세 번 해요. 하기 전에 **예상**을 적고, 해 본 뒤 **기록표**를 채우고**왜 그럴까**를 읽어요. `실행` 처럼 네모 친 말은 화면의 버튼 이름이에요.

정리 · 평가 세 결과를 비교해 **기억할 것**과 **현장에서는**을 확인하고, 문항지로 평가해요.

참고 자료 Corke, P., Robotics, Vision and Control: Fundamental Algorithms in Python, 3rd ed., Springer, 2023 (카메라 모델·보정·색·시각 서보) · Hartley, R. & Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004 (호모그래피·DLT) · Zhang, Z., "A flexible new technique for camera calibration", IEEE Transactions on Pattern Analysis and Machine Intelligence, 22(11), 2000 · Tsai, R. & Lenz, R., "A new technique for fully autonomous and efficient 3D robotics hand/eye calibration", IEEE Transactions on Robotics and Automation, 5(3), 1989 · Gonzalez, R. & Woods, R., Digital Image Processing, 4th ed., Pearson, 2018 (색 공간·임계값·연결 요소) · Chaumette, F. & Hutchinson, S., "Visual servo control, Part I: Basic approaches", IEEE Robotics & Automation Magazine, 13(4), 2006

24차시 7-1 카메라와 좌표

Camera and Coordinates · 단원 안 1 / 3차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

사진 속 위치를 (u, v) 로 읽을 수 있다. **픽셀**만으로는 책상 위 mm 를 알 수 없다. 그 까닭을 설명할 수 있다.

위치를 아는 **마커** 4개로 보정을 할 수 있다. 보정은 **픽셀**을 책상 위 mm 로 바꾼다. 이 변환을 **호모그래피** H(호모그래피) 라고 한다. 보정 뒤에도 남는 오차의 원인(**시차**)을 말할 수 있다.

손에 단 카메라는 손-눈 변환이 더 필요하다. 그래야 본 위치를 책상 좌표로 바꾼다. 그 까닭을 실험 결과로 설명할 수 있다.

Q 생각 열기

카메라가 본 "사진 속 위치" 가 있다. 이것을 로봇이 갈 "책상 위 위치" 로 어떻게 바꿀까?

함께 생각해 보기

사진 속 블록 위치를 로봇에게 그대로 알려 주면 로봇이 블록을 집을 수 있을까?

로봇은 위치를 무슨 단위로 알아들을까?

해 볼 것

- ☐ 사진 한 장에서 물건 하나를 골라 왼쪽 위에서 가로·세로 몇 칸째인지 어림해 본다
- ☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

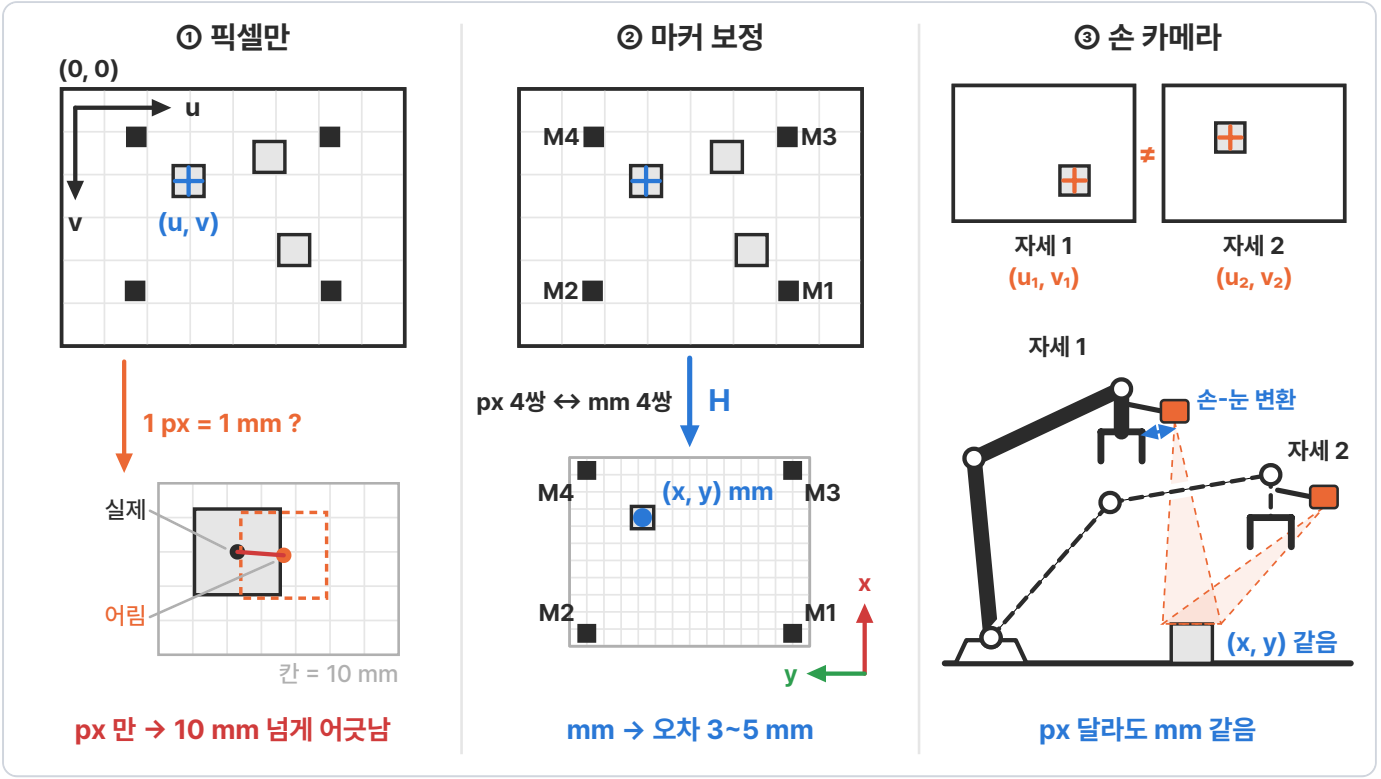


그림 7-1 사진 속 픽셀 (u, v) 를 책상 위 (x, y) mm 로 바꾸기: 픽셀만 · 마커 4개로 보정(H) · 손 카메라 (고정 카메라 높이 620 mm)
u 가로 픽셀 v 세로 픽셀 H 호모그래피 x 앞쪽 거리 y 왼쪽 거리

정의

카메라는 위치를 픽셀 (u (가로 픽셀) , v (세로 픽셀)) 로 말한다. 로봇은 mm 로 말한다. 둘 사이의 관계를 재서 맞추는 일이 카메라 보정이다. 손에 단 카메라는 손-눈 보정이 더 필요하다. 손끝에서 카메라까지의 관계를 재는 일이다.

원리

사진은 칸(**픽셀**)으로 위치를 말한다. 왼쪽 위가 (0, 0) 이다. 오른쪽으로 u, 아래로 v 가 커진다. 1 픽셀이 몇 mm 인지는 카메라 높이에 따라 다르다.

책상 위 위치를 아는 **마커** 4개의 **픽셀**을 찾는다. 그러면 픽셀을 mm 로 바꾸는 H 가 정해진다.

손에 단 카메라는 팔과 함께 움직인다. 그래서 손끝 포즈에 손-눈 변환을 곱한다. 그래야 책상 좌표가 나온다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
u	유	**가로 픽셀**: 사진 왼쪽 끝에서 오른쪽으로 몇 번째 칸인지	px
v	브이	**세로 픽셀**: 사진 위쪽 끝에서 아래로 몇 번째 칸인지	px
x	엑스	**앞쪽 거리**: 책상 위 위치. 로봇 바닥 중심에서 앞으로 떨어진 거리	mm
y	와이	**왼쪽 거리**: 책상 위 위치. 로봇 바닥 중심에서 왼쪽으로 떨어진 거리(오른쪽이면 음수)	mm
H	에이치	**호모그래피**: 픽셀 (u, v) 를 책상 (x, y) mm 로 바꾸는 3×3 표. 마커 4개로 정한다	없음
f	에프	**초점거리**: 렌즈 중심에서 사진 면까지의 거리를 픽셀로 잰 값. 클수록 크게(좁게) 찍힌다	px
Z	제트	**깊이**: 카메라에서 물체까지 바라보는 방향으로 잰 거리. 멀수록 작게 찍힌다	mm

개념 자세히

카메라는 3D 세상을 평평한 사진 한 장으로 찍는다. 사진은 가로 320칸, 세로 240칸처럼 작은 칸(**픽셀**)으로 되어 있고, 사진 속 위치는 "가로 몇 번째, 세로 몇 번째 칸" 인 (u , v) 로 말한다. 원점은 사진 왼쪽 위이고, 가로 픽셀 u 는 오른쪽으로, 세로 픽셀 v 는 아래로 커진다. 그런데 로봇은 픽셀을 모른다. 로봇이 알아듣는 것은 1 기초 로봇공학에서 배운 기준 좌표계의 mm, 곧 책상 위 위치 (x , y) 다. 그래서 카메라를 쓰려면 먼저 "이 픽셀은 책상의 어디인가" 를 알아내야 한다. 시뮬레이터의 고정 카메라는 책상 위 620 mm 에서 내려다보며, 책상 가운데 근처에서 1 픽셀이 약 1.3 mm 를 덮는다. (실제 AI 키트의 위 카메라는 보정판 위 약 368 mm 에 있어 1 픽셀이 덮는 넓이와 **시차**가 이 숫자와 다르다. 키트 프로그램은 ArUco **마커** 2개 사이의 픽셀 수로 "1 px 가 몇 mm 인지" 배율 하나와 가운데만 잰다. 카메라가 똑바로 내려다본다고 보고 쓰는 간단한 보정이다.)

사진이 만들어지는 방식은 바늘구멍 사진기(핀홀 모델)로 설명한다. 물체에서 나온 빛이 렌즈 중심 한 점을 지나 사진 면에 닿는다. 그래서 멀리 있는 물체는 작게, 가까운 물체는 크게 찍힌다. 같은 카메라라도 높이 올리면 1 **픽셀**이 책상의 더 넓은 곳을 덮는다. 렌즈에서 사진 면까지의 거리를 픽셀로 잰 값을 초점거리 f , 카메라에서 물체까지의 거리를 깊이 Z 라고 하면, 1 픽셀은 대략 $Z \div f$ mm 를 덮는다. 시뮬레이터의 고정 카메라는 f 가 약 470 px 라서 620 ÷ 470 ≈ 1.3 mm 다. 픽셀과 mm 의 관계를 계산으로 구하려면 초점거리와 사진 중심, 그리고 카메라의 위치·방향을 모두 알아야 한다. 앞의 두 가지를 내부 파라미터, 카메라의 위치·방향을 외부 파라미터라고 한다. 실제 렌즈는 사진을 조금 휘게 만들기도 해서(렌즈 왜곡), 내부 파라미터를 잴 때 이 휨도 함께 잰다. 이것들을 하나하나 재기는 번거롭다.

책상처럼 평평한 면 위의 점만 다룬다면 더 쉬운 방법이 있다. 책상 위 위치를 이미 아는 표시(**마커**) 4개를 놓고, 사진에서 그 마커들의 **픽셀**을 찾는다. 픽셀 4쌍과 mm 4쌍을 맞추면, 사진의 어느 점이든 책상 mm 로 바꾸는 3×3 표 하나가 정해진다. 이 표를 **호모그래피** H 라고 한다. 다만 이 표는 책상 면에만 맞는다. 블록 윗면은 책상보다 25 mm 높아서, 같은 표로 바꾸면 카메라 바로 아래에서 먼 블록일수록 몇 mm(이 화면에서 3~5 mm) 바깥쪽으로 어긋난다. 높이가 달라서 생기는 이 어긋남을 **시차**라고 한다.

카메라를 로봇 손에 달면(손 카메라) 물체를 가까이에서 볼 수 있지만, 팔이 움직일 때마다 카메라도 움직여 같은 블록의 **픽셀**이 계속 바뀐다. 이때는 "손끝이 지금 어디에 있는가"(2 기구학 & 역기구학의 순기구학)와 "카메라가 손끝의 어디에 붙어 있는가"(손-눈 변환)를 곱해서, 카메라가 본 것을 책상 좌표로 옮긴다. 손-눈 변환을 재는 일을 **손-눈 보정**이라고 한다. 시뮬레이터의 손 카메라는 플랜지 축에서 44.4 mm 옆, 플랜지 면에서 11.6 mm 아래에 붙어 있어서, 팔 자세가 달라도 같은 블록의 책상 좌표가 거의 같게 나온다.

대학 · 수식

핀홀 모델: $u = f_x \cdot X/Z + c_x$, $v = f_y \cdot Y/Z + c_y$ (X, Y, Z: 카메라 좌표 mm, f: 초점거리 px, c: 사진 중심 px)

1 픽셀이 덮는 길이 ≈ Z / f → 카메라가 2배 멀어지면 2배

호모그래피: $[x, y, 1]^T \propto H\,[u, v, 1]^T$, H 는 3×3 이고 배율을 빼면 미지수 8개 → 점 하나가 식 2개를 주므로 점 4개로 결정

시차: 카메라 높이 h_c, 물체 높이 h, 카메라 바로 아래 점에서 거리 d 이면 책상 면 호모그래피의 오차 ≈ $d \cdot h / (h_c - h)$

손 카메라: T_바닥←물체 = T_바닥←손끝 · T_손끝←카메라 · T_카메라←물체, 손-눈 변환 T_손끝←카메라 는 여러 자세에서 같은 마커를 찍어 AX = XB 꼴의 식을 풀어 구한다

탐구 활동 시뮬레이터 [교육] 탭 · 7-1 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 픽셀만

처음 조건 · 픽셀 → mm 방법: 1 px = 1 mm 어림

목표 사진 속 위치를 **픽셀**로 읽고, 픽셀을 그대로 mm 로 어림하면 얼마나 틀리는지 알아본다.

예상 블록의 **픽셀**을 "1 px = 1 mm" 로 바꿔 책상 위치를 어림하면, 실제 위치와 몇 mm 쯤 차이 날까?

☐ 1. 1 mm 안 ☐ 2. 3~5 mm ☐ 3. 10 mm 넘게

할 일 고정 카메라 화면에서 블록 하나(예: 빨강)를 누른다. 그리고 **픽셀** (u (가로 픽셀), v (세로 픽셀)) 를 읽는다.

볼 것 왼쪽 위가 (0, 0) 이고 u 는 오른쪽, v 는 아래로 커진다. "1 px = 1 mm" 로 어림한 위치는 실제와 10 mm 넘게 어긋난다

왜 **픽셀**은 사진 속 칸 번호라서 단위가 mm 가 아니다. 이 카메라는 620 mm 위에서 내려다봐 1 px 가 약 1.3 mm(620 ÷ 470)를 덮으므로, "1 px = 1 mm" 로 보면 사진 가운데에서 멀수록 실제보다 덜 간 위치가 나온다. 빨강 블록은 가로로 43 px 떨어져 보이지만 실제로는 55 mm 떨어져 있어 약 12 mm 어긋난다. 사진 가운데가 책상의 어디인지(카메라 방향)도 어림이라, 다른 블록은 더 어긋날 수 있다(파랑 약 25 mm).

핵심 **1 px = 1 mm 어림 → 10 mm 넘게 어긋남**

2 활동 2 · 마커 보정

바꾸는 것 · 픽셀 → mm 방법: 1 px = 1 mm 어림 → 마커 4개로 보정

목표 **마커** 4개로 **픽셀** → mm 변환을 구하고, 보정한 뒤 블록 위치 오차가 얼마나 줄어드는지 알아본다.

예상 위치를 아는 **마커** 4개로 보정하면 블록 위치 오차가 0 이 될까, 아니면 조금 남을까?

☐ 1. 0 이 된다 ☐ 2. 몇 mm 남는다 ☐ 3. 더 커진다

할 일 보정 을 누른 뒤 같은 블록을 다시 누른다

볼 것 **마커** 4개의 px ↔ mm 표가 나오고, 블록의 책상 위치 (x (앞쪽 거리), y (왼쪽 거리)) 가 실제와 약 3~5 mm 차이로 맞으며 팔이 그 위 40 mm 로 간다

왜 **마커** 4쌍(**픽셀** ↔ mm)이 **호모그래피** H (호모그래피) 의 미지수 8개를 모두 정하므로, 책상 면 위의 점은 거의 정확히 mm 로 바뀐다. 그러나 블록 윗면은 책상보다 25 mm 높아서, 책상 면에 맞춘 H 로 바꾸면 카메라 바로 아래 점에서 바깥쪽으로 밀린 위치가 나온다. 빨강 블록은 카메라 바로 아래 점에서 약 90 mm 떨어져 있어 90 × 25 ÷ (620 − 25) ≈ 3.8 mm 어긋나며, 이것이 남은 오차(**시차**)다.

핵심 **마커 4개로 보정 → 오차가 3~5 mm 로 준다**

3 활동 3 · 손 카메라

바꾸는 것 · 카메라: 고정 카메라 → 손 카메라 (자세 3개)

목표 팔 자세를 바꿔 가며 손 카메라로 같은 블록을 보고, **픽셀**과 책상 좌표가 각각 어떻게 달라지는지 알아본다.

예상 팔 자세를 바꾸면 사진 속 블록의 **픽셀**은 어떻게 될까? 그 픽셀로 구한 책상 위치도 함께 바뀔까?

☐ 1. 픽셀은 바뀌고 책상 위치는 거의 같다 ☐ 2. 픽셀과 책상 위치가 함께 바뀐다 ☐ 3. 픽셀과 책상 위치가 모두 그대로다

할 일 자세 1 → 자세 2 → 자세 3 을 차례로 누른다. 손 카메라로 빨강 블록을 본다.

볼 것 자세마다 블록의 **픽셀**은 바뀌지만, 손-눈 변환으로 구한 책상 위치는 거의 같다(실제와 1 mm 안)

왜 손 카메라는 팔과 함께 움직이므로 자세마다 블록이 사진의 다른 칸에 찍힌다. 하지만 손끝 포즈(순기구학)에 손-눈 변환(플랜지 축에서 44.4 mm 옆, 플랜지 면에서 11.6 mm 아래)을 곱하면 그 순간 카메라가 어디에서 어느 쪽을 보는지 알 수 있다. 카메라에서 블록 **픽셀** 쪽으로 그은 선이 블록 윗면 높이(25 mm)와 만나는 점은 자세와 상관없이 같으므로, 세 자세의 책상 위치가 실제와 1 mm 안(이 화면에서 약 0.3 mm)으로 모인다. 시뮬레이터는 손-눈 변환과 팔의 포즈를 정확히 알기 때문이다. 실물에서는 **손-눈 보정**의 오차와 팔의 정확도 오차가 더해져 몇 mm 가 될 수 있다.

핵심 **자세마다 픽셀은 달라도 → 책상 위치는 1 mm 안**

✎ 기록표: 화면의 결과를 적어요

활동	위치 단위(px · mm · mm)	블록 위치 오차(mm)	카메라(고정 · 고정 · 손)
활동 1 픽셀만			
활동 2 마커 보정			
활동 3 손 카메라			

정리: 세 활동 비교하기

세 경우의 블록 위치 오차를 표로 놓는다. 보정 없이 · 보정 후 · 손 카메라다. 무엇을 더해야 픽셀이 mm 가 되는지 본다.

알게 된 것:

정리 질문

카메라가 본 픽셀을 로봇이 갈 mm 로 바꾸려면 무엇이 필요할까?

내 답:

기억할 것

픽셀은 사진 속 칸 번호일 뿐이다. 마커로 보정해야 mm 가 된다. 손 카메라는 손-눈 변환까지 곱해야 한다.

현장에서는

컨베이어 위 부품을 카메라로 찾아 집는 로봇이 있다. 이 로봇은 작업 전에 보정판을 찍어 카메라 보정을 한다. 보정판은 위치를 아는 체커보드나 마커다. 그 결과로 사진 속 부품 위치를 로봇 좌표로 바꾼다. 카메라를 다시 달면 픽셀과 mm 의 관계가 바뀐다. 받침대가 조금만 움직여도 바뀐다. 그래서 현장에서는 보정 결과를 정기적으로 확인한다. 필요하면 다시 보정한다.

교사용 답

예상 정답: 활동 1: 3. 10 mm 넘게 · 활동 2: 2. 몇 mm 남는다 · 활동 3: 1. 픽셀은 바뀌고 책상 위치는 거의 같다

정리 질문: 고정 카메라는 위치를 아는 마커 4개로 보정해야 하고, 손 카메라는 손끝 포즈에 손-눈 변환까지 곱해야 한다.

개념 확인 질문: 픽셀 (u, v) 와 책상 위치 (x, y) 는 무엇이 다를까?

다음 차시 · 25차시 7-2 색으로 찾기: 카메라는 사진 속 빨간 블록을 어떻게 찾을까? 방이 어두워지면 왜 놓칠까?

픽셀 pixel — 사진을 이루는 작은 점. 사진 속 위치는 (가로 칸, 세로 칸)으로 말한다.

카메라 보정 camera calibration — 사진의 픽셀 위치와 실제 위치(mm)의 관계를 재서 맞추는 것.

호모그래피 Homography — 한 평면 위의 점을 사진 픽셀과 이어 주는 3×3 변환. 위치를 아는 점 4개로 정한다

손-눈 보정 hand-eye calibration — 로봇 손에 단 카메라가 손끝에서 어디에, 어느 방향으로 붙어 있는지 재는 것.

핀홀 카메라 모델 Pinhole camera model — 빛이 렌즈 중심 한 점을 지나 사진 면에 닿는다고 보는 카메라 모델. u = f·X/Z + c

마커 fiducial marker (ArUco) — 카메라가 위치를 쉽게 찾도록 만든 흑백 무늬 표.

시차 Parallax — 높이가 다른 점을 같은 평면으로 보고 바꿀 때 생기는 위치 어긋남

25차시 7-2 색으로 찾기

Color Detection · 단원 안 2 / 3차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

임계값으로 **마스크**를 만들 수 있다. 색상 범위를 조절해 찾으려는 물체만 남길 수 있다.

조명이 바뀌면 RGB 기준과 HSV 기준의 결과가 다르다. 그 까닭을 R(빨강 값) · H(색상) · V(밝기) 값으로 설명할 수 있다.

마스크 덩어리의 **무게중심**으로 위치를 구할 수 있다. 가림이 위치 오차를 만드는 까닭을 설명할 수 있다.

Q 생각 열기

카메라는 사진 속 빨간 블록을 어떻게 찾을까? 방이 어두워지면 왜 놓칠까?

함께 생각해 보기

사진에서 빨간 블록을 찾으라고 하면 컴퓨터는 무엇을 볼까?

방이 어두워져도 빨간색은 빨간색으로 보일까?

해 볼 것

☐ 주변에서 빨강과 비슷한 색(주황·분홍) 물건을 찾아 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

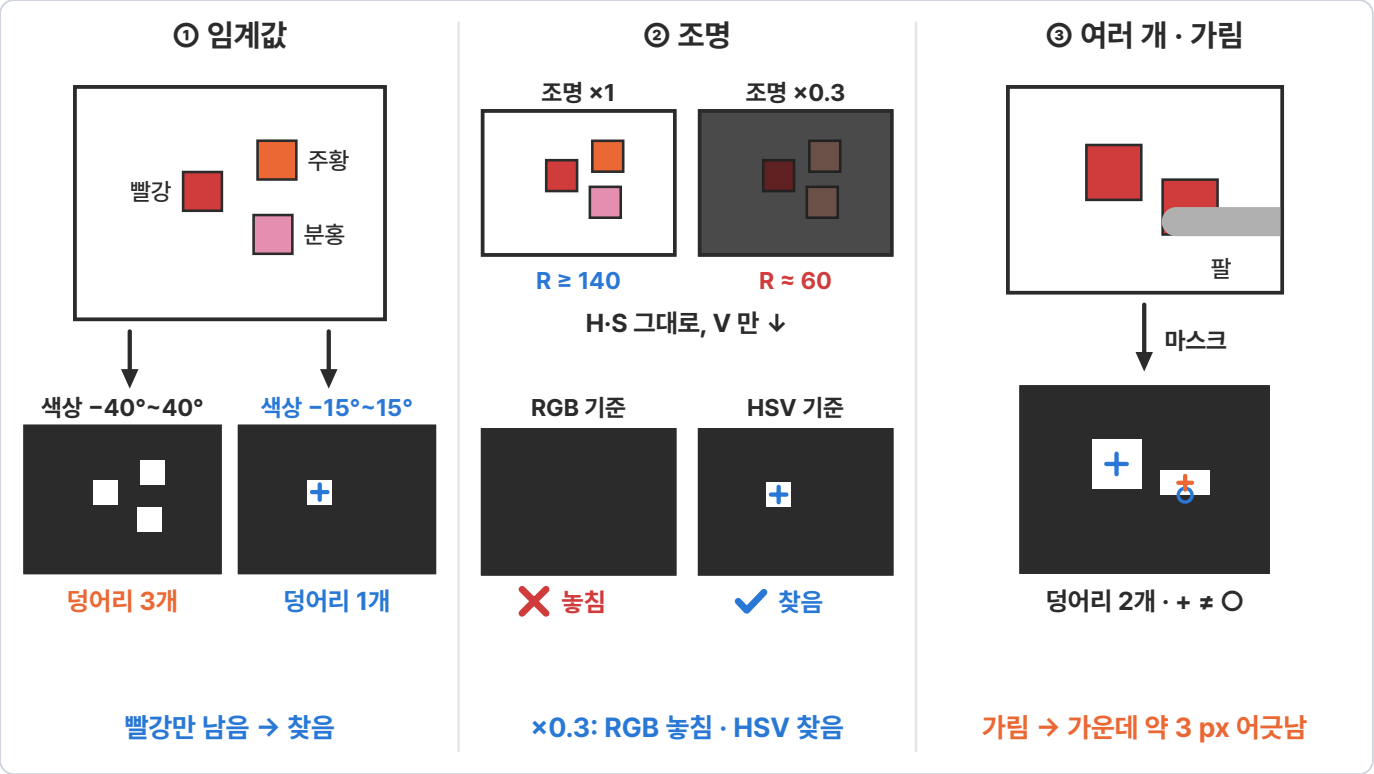


그림 7-2 색으로 빨강 블록 찾기: 색상 범위 좁히기 · 조명 ×0.3 에서 RGB 기준과 HSV 기준 · 팔이 가린 블록의 가운데 R 빨강 값 H 색상 S 채도 V 밝기

정의

색 찾기는 먼저 마스크를 만든다. 픽셀마다 색이 정해 둔 범위(임계값) 안인지 본다. 안이면 1, 밖이면 0 으로 칠한다. 1 이 모인 덩어리의 가운데를 물체 위치로 쓴다. 이 가운데를 무게중심이라 한다.

원리

색을 색상(H)·채도(S)·밝기(V)로 나눈다. 그러면 조명이 어두워져도 색상은 거의 그대로다.

임계값 범위가 너무 넓으면 비슷한 색까지 잡는다. 너무 좁으면 물체를 놓친다.

떨어진 물체는 덩어리가 따로 잡힌다. 가려지면 보이는 부분만 잡혀 가운데가 어긋난다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
R	알	**빨강 값**: 픽셀의 빨간 빛 세기(0~255)	없음
G	지	**초록 값**: 픽셀의 초록 빛 세기(0~255)	없음
B	비	**파랑 값**: 픽셀의 파란 빛 세기(0~255)	없음
H	에이치	**색상**: 어떤 색인지 각도로 적은 값. 이 화면은 빨강 = 0°, −180°~180°	°
S	에스	**채도**: 색이 얼마나 선명한지(0 = 회색, 1 = 가장 선명)	없음
V	브이	**밝기**: 픽셀이 얼마나 밝은지. 어두워지면 이것만 줄어든다	없음
u	유	**가로 픽셀**: 사진 왼쪽 끝에서 오른쪽으로 몇 번째 칸인지	px
v	브이	**세로 픽셀**: 사진 위쪽 끝에서 아래로 몇 번째 칸인지	px

개념 자세히

디지털 사진의 **픽셀**은 빨강 값 R(빨강 값), 초록 값 G(초록 값), 파랑 값 B(파랑 값) 세 숫자(0~255, 줄여서 RGB)로 색을 적는다. "빨간 블록을 찾아라" 를 가장 쉽게 쓰면 "R 이 크고 G·B 는 작은 픽셀" 이다. 시뮬레이터의 RGB 기준은 $R \geq 140$, $G \leq 90$, $B \leq 90$ 이다. 기준 안에 든 픽셀은 하양(1), 나머지는 검정(0)으로 칠한 그림을 **마스크**라고 하고, 이렇게 나누는 기준 값을 **임계값**이라고 한다.

문제는 조명이다. 방이 어두워지면 같은 빨간 블록도 R·G·B 세 값이 함께 줄어든다. 조명을 ×0.3 으로 낮추면 블록의 R 은 약 60 으로 떨어져 RGB 기준으로는 블록을 놓친다. 그래서 색을 색상 H(어떤 색인가)·채도 S(얼마나 선명한가)·밝기 V(얼마나 밝은가)로 나눈 HSV 를 쓴다. 어두워지면 V 만 줄고 H 와 S 는 거의 그대로라서, 색상으로 정한 기준은 조명이 바뀌어도 오래 버틴다. 이 화면은 색상을 빨강 = 0° 로 두고 −180°~180° 로 적는다. 빨강 블록은 약 0°, 주황 물체는 약 29°, 분홍 물체는 약 −25° 다.

마스크에서 하얀 **픽셀**이 붙어 있는 덩어리 하나를 물체 하나로 본다. 덩어리의 가운데는 하얀 픽셀들의 가로 픽셀 u(가로 픽셀) 평균과 세로 픽셀 v(세로 픽셀) 평균, 곧 **무게중심**이다. 아주 작은 덩어리는 잡음으로 보고 버린다(이 화면은 25 픽셀 미만). 블록이 둘이면 덩어리도 둘이다. 그런데 팔이 블록 일부를 가리면 보이는 부분만 하얗게 남아, 무게중심이 보이는 쪽으로 쏠린다. 이 화면의 위 카메라(7-1 과 다른 카메라, 책상 위 520 mm)에서 1 px 는 블록 윗면 기준 약 1.2 mm 이므로, 몇 px 쏠리면 집을 때 그만큼 mm 로 빗나간다.

색 찾기는 계산이 가볍고 빨라서 원리를 익히고 빠르게 시험하기 좋다. 하지만 비슷한 색, 그림자, 가림에 약하다. 그래서 실제 로봇에서는 학습된 물체 검출(YOLO 계열 등)을 함께 쓰고, 이 앱의 카메라 탭에도 YOLO 검출이 들어 있다. 어떤 방법으로 찾든 결과는 **픽셀**이므로, 7-1 의 보정으로 mm 로 바꿔 로봇에게 넘긴다. 8 로봇 학습의 캡스톤에서 이 두 단계를 이어 쓴다.

대학 · 수식

마스크(u, v) = 1 (H_min ≤ H(u, v) ≤ H_max 이고 S(u, v) ≥ S_min 일 때), 아니면 0

무게중심: ū = Σ u·마스크 / Σ 마스크, v̄ = Σ v·마스크 / Σ 마스크

V = max(R, G, B), S = (max − min) / max, H 는 가장 큰 값이 어느 색인지와 나머지 두 값의 차로 정한다

조명 ×k 로 (R, G, B) → (kR, kG, kB) 이면 H 와 S 는 그대로, V 만 k 배 (값이 0~255 안에 있을 때)

탐구 활동 시뮬레이터 [교육] 탭 · 7-2 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 임계값 — 처음 조건 · 색상 범위: −40°~40°

목표	색상 범위와 채도 기준을 바꿔 가며 빨강 블록만 **마스크**에 남기는 **임계값**을 찾는다.
예상	색상 범위가 −40°~40° 로 넓으면 **마스크**에 덩어리가 몇 개 잡힐까? 범위를 좁히면 어떻게 될까? ☐ 1. 1개 ☐ 2. 2개 ☐ 3. 3개
할 일	H 최소 · H 최대 · S 최소 슬라이더를 움직인다. 빨강 블록만 하얗게 남긴다.
볼 것	처음(−40°~40°)에는 주황·분홍까지 덩어리 3개가 잡히고, 색상 범위를 −15°~15° 정도로 좁히면 덩어리 1개, 결과 "찾음"
왜	주황 물체(약 29°)와 분홍 물체(약 −25°)의 색상은 −40°~40° 안에 들어 빨강 블록(약 0°)과 함께 하얗게 남는다. 범위를 −15°~15° 로 좁히면 두 물체는 범위 밖으로 나가고 빨강 블록만 남아 덩어리가 1개가 된다. 다만 범위를 0° 하나로만 좁히면 조명과 그림자로 색상이 몇 도씩 달라진 블록 **픽셀**까지 빠져 놓칠 수 있으므로, 찾으려는 색상 앞뒤로 여유를 둔다.
핵심	**범위를 −15°~15° 로 좁힘 → 덩어리 3개에서 1개**

2 활동 2 · 조명 — 바꾸는 것 · 조명: 보통 밝기 → ×0.3

목표	같은 장면을 어둡게 했을 때 RGB 기준과 HSV 기준 중 어느 쪽이 블록을 계속 찾는지 비교한다.
예상	조명을 ×0.3 까지 낮추면 R(빨강 값) ≥ 140 으로 찾는 RGB 기준과 색상으로 찾는 HSV 기준은 각각 블록을 찾을까? ☐ 1. RGB 만 찾는다 ☐ 2. HSV 만 찾는다 ☐ 3. 둘 다 찾는다 ☐ 4. 둘 다 놓친다
할 일	같은 장면에서 조명 슬라이더를 ×0.3 까지 내린다. 그다음 RGB기준 과 HSV기준 을 번갈아 누른다.
볼 것	RGB 기준(R ≥ 140)은 블록을 놓치고, HSV 기준(색상 −15°~15°, S(채도) ≥ 0.45)은 그대로 찾는다
왜	조명이 ×0.3 이 되면 블록의 (R(빨강 값), G(초록 값), B(파랑 값)) = (205, 35, 40) 이 약 (62, 11, 12) 로 세 값이 함께 줄어, R ≥ 140 기준을 넘지 못한다. 그러나 색상 H(색상) 와 채도 S 는 세 값의 비율로 정해지므로 그대로(H 약 −2°, S 약 0.83)이고, 밝기 V(밝기) 만 0.3 배가 된다. 그래서 색상과 채도로 정한 HSV 기준은 같은 블록을 계속 찾는다.
핵심	**조명 ×0.3 → RGB 는 놓치고 HSV 는 찾음**

3 활동 3 · 여러 개 · 가림 — 바꾸는 것 · 장면: 블록 1개 → 블록 2개 (하나는 팔이 가림)

목표	블록이 여러 개이거나 일부가 가려졌을 때 **마스크** 덩어리와 **무게중심**이 어떻게 되는지 알아본다.
예상	빨강 블록 2개 중 하나를 팔이 일부 가리면, 덩어리 수와 그 블록의 가운데(+)는 어떻게 될까? ☐ 1. 덩어리 1개, + 는 그대로 ☐ 2. 덩어리 2개, + 는 그대로 ☐ 3. 덩어리 1개, + 가 어긋남 ☐ 4. 덩어리 2개, + 가 어긋남
할 일	빨강 블록 2개가 놓인 장면에서 팔로 가리기 를 누른다
볼 것	덩어리가 블록마다 따로 잡히고, 팔이 가린 블록의 + 가 진짜 가운데(○)에서 약 3 px(≈ 4 mm) 어긋난다
왜	떨어져 있는 두 블록은 하얀 **픽셀**이 서로 붙지 않아 덩어리 2개로 따로 잡힌다. 가린 블록은 보이는 부분의 픽셀만 남으므로, 그 평균인 **무게중심**이 보이는 쪽으로 쏠려 진짜 가운데에서 약 3 px 어긋난다. 이 카메라에서 1 px 는 블록 윗면 기준 약 1.2 mm 이므로, 이 위치로 집으러 가면 약 3.7 mm 빗나간다.
핵심	**팔이 가림 → + 가 약 3 px(≈ 4 mm) 어긋남**

✎ 기록표: 화면의 결과를 적어요

활동	조건(보통 밝기 · 가장 어둡게 · 가림)	결과(찾음 · RGB 못 찾음 / HSV 찾음 · 덩어리 수)	위치 오차(px, ≈ mm)
활동 1 임계값			
활동 2 조명			
활동 3 여러 개 · 가림			

정리: 세 활동 비교하기

세 조건의 마스크를 나란히 놓는다. 결과와 위치 오차를 표로 본다. 색 찾기가 어디에 강하고 약한지 정리한다.

알게 된 것:

정리 질문

색으로 찾기는 어디에 강하고 어디에 약할까?

내 답:

기억할 것

색은 HSV 의 색상으로 찾으면 조명에 강하다. 그래도 비슷한 색과 가림에는 약하다.

현장에서는

과일 · 농산물 선별기는 이 방법을 널리 쓴다. 포장 라인의 검사 카메라도 마찬가지다. 색으로 종류를 나누거나 불량품을 골라낸다. 조명이 바뀌면 결과가 흔들릴 수 있다. 그래서 검사 구역을 가리개로 덮는다. 그리고 일정한 조명을 비추는 경우가 많다. 모양이 복잡하거나 서로 겹친 물체도 있다. 이때는 학습된 물체 검출을 함께 쓴다.

교사용 답

예상 정답: 활동 1: 3. 3개 · 활동 2: 2. HSV 만 찾는다 · 활동 3: 4. 덩어리 2개, + 가 어긋남

정리 질문: 색상 H 로 찾으면 조명이 바뀌어도 잘 찾지만, 비슷한 색과 가림에는 약하다.

개념 확인 질문: 마스크에서 하얀 픽셀과 검은 픽셀은 각각 무엇을 뜻할까?

다음 차시 · 26차시 7-3 시각 서보: 블록이 움직인다. 카메라로 보면서 따라가려면 어떻게 해야 할까?

임계값
threshold
"이 값보다 크면 빨강" 처럼 나누는 기준 값.

HSV 색 공간
HSV color space
색을 색상(H)·채도(S)·밝기(V)로 나눠 적는 방법. 조명이 바뀌어도 색상은 거의 그대로다

무게중심
Centroid
덩어리 픽셀들의 u 평균과 v 평균. 물체 위치로 쓴다

마스크
Mask
조건에 맞는 픽셀은 1(하양), 아닌 픽셀은 0(검정)으로 칠한 그림

연결 요소
Connected component
마스크에서 서로 붙어 있는 1 픽셀들의 덩어리. 물체 하나로 본다

픽셀
pixel
사진을 이루는 작은 점. 사진 속 위치는 (가로 칸, 세로 칸)으로 말한다.

26차시 7-3 시각 서보

Visual Servoing · 단원 안 3 / 3차시 · 50분 (도입 5 · 개념 10 · 활동 8 × 3 · 정리 6 · 평가 5)

학습 목표

한 번 보고 이동하는 방식이 있다. 계속 보며 이동하는 **시각 서보**도 있다. 둘의 차이를 평균 추적 **오차**로 비교할 수 있다.

게인 $K_{\text{게인}}$ 와 사진 지연 $\tau_{\text{지연}}$ 의 영향을 설명할 수 있다. 추적 **오차**와 흔들림에 주는 영향이다. 흔들림이 커지기 시작하는 K 의 한계를 구할 수 있다.

$L_{\text{이미지 자코비안}}$ 이 하는 일을 설명할 수 있다. L 은 손끝 움직임(mm)을 사진 속 움직임(px)으로 바꾼다. 그 역 L^{-1} 은 사진 속 **오차**를 손끝 움직임으로 바꾼다.

Q 생각 열기

블록이 움직인다. 카메라로 보면서 따라가려면 어떻게 해야 할까?

함께 생각해 보기

굴러가는 공을 한 번만 보고 눈을 감은 채 뛰어가면 잡을 수 있을까?

공을 끝까지 보면서 손을 움직이면 무엇이 다를까?

해 볼 것

☐ 짝이 천천히 옮기는 펜 끝을 한 번 보고 눈을 감은 채 손가락으로 짚어 본다

☐ 질문의 답을 한 줄로 예상해 적는다

내 생각:

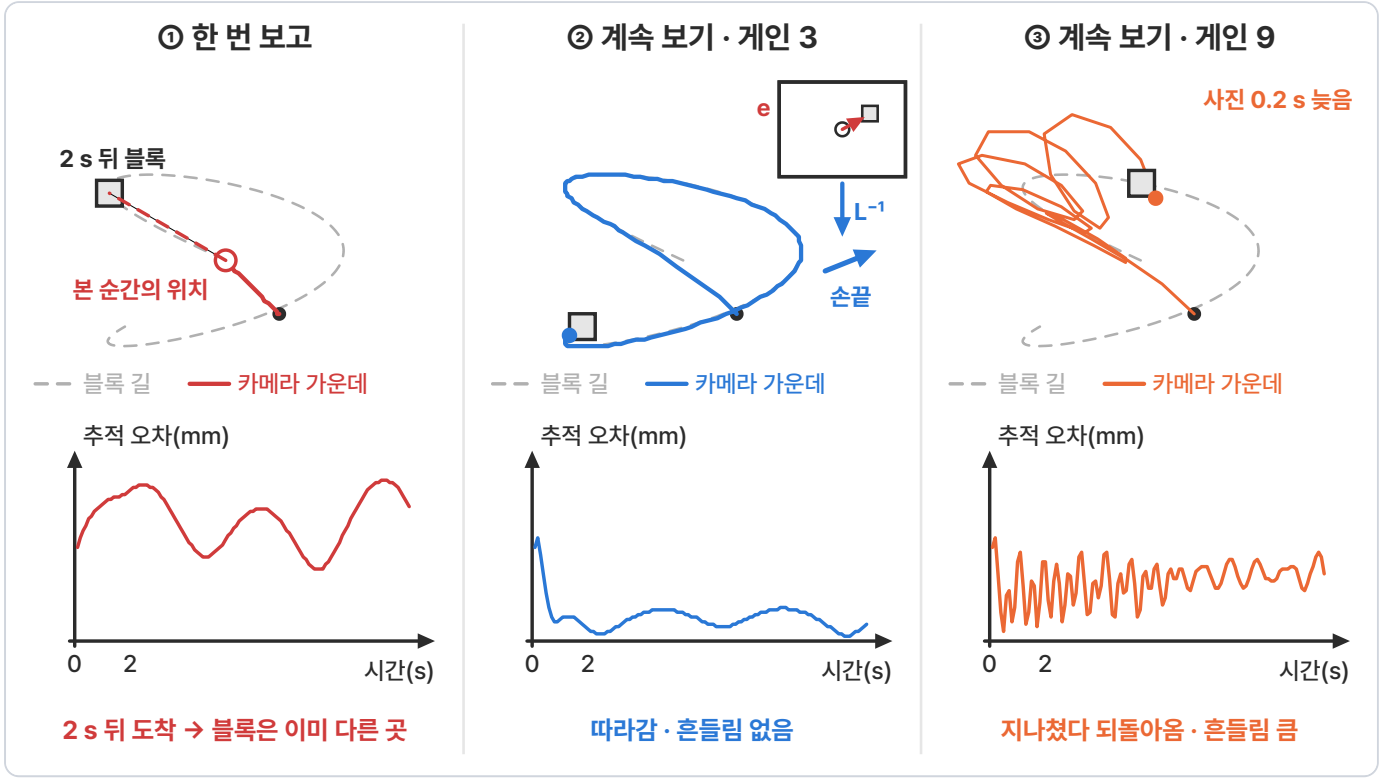


그림 7-3 움직이는 블록 따라가기: 한 번 보고 · 계속 보기 게인 3 · 게인 9 (사진 지연 0.2 s), 위에서 본 카메라 가운데의 자취와 추적 오차 e 사진 속 오차 L 이미지 자코비안

정의

시각 서보는 피드백 제어다. 카메라로 목표와의 차이(사진 속 오차)를 매 순간 잰다. 그리고 그 차이를 줄이는 쪽으로 로봇을 움직인다.

원리

한 번 보고 이동: 본 순간의 위치로 간다. 그동안 목표가 움직이면 빗나간다.

계속 보며 이동: 매 프레임 사진 속 **오차**를 본다. 그 반대쪽으로 조금씩 움직인다. **게인** K 가 작으면 뒤처진다. 너무 크면 흔들린다.

이미지 자코비안 L 은 두 움직임을 잇는 표다. 손끝 움직임(mm)과 사진 속 점의 움직임(px)이다. 그 역 L^{-1} 을 쓰면 사진 속 **오차**를 손끝을 옮길 방향과 거리로 바꿀 수 있다.

Σ 이 주제의 기호

기호	읽기	뜻	단위
e	이	**사진 속 오차**: 블록이 사진 가운데에서 벗어난 픽셀 수(가로, 세로)	px
K	케이	**게인**: 오차를 얼마나 세게 고칠지 곱하는 수. 작으면 뒤처지고, 너무 크면 흔들린다	/s
L	엘	**이미지 자코비안**: 손끝을 1 mm 옮길 때 블록이 사진에서 몇 px 움직이는지 적은 2×2 표	px/mm
τ	타우	**지연**: 사진을 찍고 처리하느라 늦게 들어오는 시간. 4단원의 토크 τ 와 다른 뜻이다	s
f	에프	**초점거리**: 렌즈 중심에서 사진 면까지의 거리를 픽셀로 잰 값	px
Z	제트	**깊이**: 카메라에서 블록 윗면까지의 거리	mm

개념 자세히

7-1 과 7-2 에서는 사진을 한 장 찍고, 블록을 찾고, mm 로 바꾼 뒤 그곳으로 갔다. 이것을 "**한 번 보고 이동**" 이라고 한다. 블록이 가만히 있고 보정이 정확하면 이것으로 충분하다. 그러나 블록이 컨베이어 위에서 움직이거나 보정이 조금 틀렸다면, 도착했을 때 손끝은 블록 위에 있지 않다. 눈을 감고 걸어가는 것과 같다. 시뮬레이터에서 처음 한 번만 보고 2 초 동안 그 자리로 가면, 도착할 때 블록은 이미 다른 곳에 있다.

시각 서보는 5 로봇 제어공학의 **피드백 제어**를 카메라로 하는 것이다. 손 카메라 사진의 가운데를 목표로 두고, 블록이 사진 가운데에서 얼마나 벗어났는지(사진 속 **오차** $e_{\text{사진 속 오차}}$, px)를 매 프레임(1/30 초) 잰다. 그리고 오차를 줄이는 쪽으로 손끝을 움직인다. 얼마나 세게 움직일지는 **게인** $K_{\text{게인}}$ 가 정한다. K = 3 이면 오차가 10 px 일 때 1 초에 30 px 를 줄이는 빠르기로 움직인다. K 가 작으면 조금씩만 고쳐 블록에 뒤처지고, K 가 크면 빨리 따라간다. 그런데 사진은 찍고 처리하는 동안 늦게 들어온다. 이 늦음을 지연 $\tau_{\text{지연}}$ 라고 하며, 이 화면에서는 0.2 초다(4 로봇 동역학의 토크 τ 와 다른 뜻이다). 늦은 사진을 보고 세게 고치면 목표를 지나쳤다가 되돌아오기를 되풀이하며 흔들린다. 이론으로는 $K \times \tau$ 가 약 1.57(π/2)을 넘으면 흔들림이 점점 커진다. 이 화면은 1/30 초마다 한 번씩 고쳐서 한계가 조금 더 낮아, K 가 약 7.2 를 넘으면 흔들림이 점점 커진다. 피드백 제어에서 게인이 너무 크면 오버슈트와 진동이 생기는 것과 같은 현상이다.

"블록이 사진에서 오른쪽으로 20 px" 라는 **오차**를 "손끝을 어느 쪽으로 몇 mm" 로 바꾸려면 둘의 관계를 알아야 한다. 손끝을 1 mm 움직일 때 블록이 사진에서 어느 방향으로 몇 px 움직이는지 적은 표가 **이미지 자코비안** $L_{\text{이미지 자코비안}}$ 이다. 3 미분기구학 · 궤적 생성의 자코비안이 관절 속도와 손끝 속도를 이었듯, 이미지 자코비안은 카메라 속도와 사진 속 점의 속도를 잇는다. 카메라가 물체에 가까울수록 같은 1 mm 이동에 사진 속 점이 더 많이 움직인다. 1 mm 당 픽셀 수는 초점거리 $f_{\text{초점거리}}$ 를 깊이 $Z_{\text{깊이}}$ 로 나눈 값이다. 이 화면의 손 카메라는 f 가 약 320 px 이고 블록 윗면에서 약 133 mm 위에 있어서($Z \approx 133$ mm), 손끝 1 mm 가 사진에서 약 2.4 px 이다.

시각 서보의 큰 장점은 보정이 조금 틀려도 된다는 것이다. 매 순간 사진으로 결과를 다시 보므로, **이미지 자코비안**이 조금 틀려도 방향만 대략 맞으면 **오차**는 줄어든다. 대신 목표가 사진 밖으로 나가면 쓸 수 없다. 이 앱 콘솔의 "색 따라가기"(손 카메라가 고른 색을 따라감)도 같은 원리로 움직인다.

대학 · 수식

사진 속 오차: $e = s - s^*$ (s: 블록이 보이는 픽셀, s^*: 목표 픽셀 = 사진 가운데)

이미지 자코비안: $\dot{s} = L\,v$ (v: 카메라 속도 mm/s, $\dot{s}$: 사진 속 점의 속도 px/s)

제어 법칙: $v = -K\,L^{-1}\,e \rightarrow \dot{e} = -K\,e$, $e(t) = e(0)\cdot\exp(-K\,t)$ (목표가 멈춰 있을 때. 이 화면은 손끝을 $x \cdot y$ 로만 옮겨 L 이 2×2 다. 6축을 다 쓰면 L 은 2×6 이고 L^{-1} 대신 의사역행렬 L^{+} 를 쓴다)

점 하나가 광축에 수직으로 움직일 때 L 의 크기 $\approx f / Z$ ($f \approx 320$ px, $Z \approx 133$ mm 이면 약 2.4 px/mm)

지연 τ 가 있으면 $\dot{e}(t) = -K\,e(t-\tau)$ 이고, $K\cdot\tau > \pi/2$ 이면 흔들림이 커지며 발산한다 (연속 시간으로 본 값. $\tau = 0.2$ s 이면 $K \approx 7.9$ /s 가 한계)

1/30 s 마다 고치는 이 화면: $e[n+1] = e[n] - K\cdot\Delta t\cdot e[n-6]$ 이고 한계는 $K = 2\sin(\pi/26) \div \Delta t \approx 7.2$ /s

탐구 활동 시뮬레이터 [교육] 탭 · 7-3 · 같은 동작을 한 가지만 바꿔 세 번

1 활동 1 · 한 번 보고

처음 조건 · 방식: 한 번 보고 가기

목표	사진을 한 번만 보고 움직이면 움직이는 블록을 얼마나 놓치는지 알아본다.
예상	처음 본 블록 자리로 2 초 동안 가면, 도착했을 때 손끝은 블록 위에 있을까? ☐ 1. 블록은 이미 다른 곳에 있다 ☐ 2. 블록 바로 위에 있다
할 일	`한 번 보고 가기` 를 누른다
볼 것	2 초 뒤 도착했을 때 블록은 이미 다른 곳에 있고, 그 뒤로도 추적 **오차**가 크게 남는다(평균 약 51 mm)
왜	이 방식은 출발 순간의 사진으로 목표를 정한 뒤 다시 보지 않는 개루프 동작이다. 2 초 동안 가는 사이 블록은 계속 움직여 다른 곳에 가 있고, 도착한 뒤에도 사진을 보지 않으니 블록이 멀어지고 가까워지는 대로 **오차**가 오르내린다. 그래서 평균 추적 오차가 약 51 mm 로 크게 남는다.
핵심	**한 번만 봄 → 평균 추적 오차 약 51 mm**

2 활동 2 · 계속 보기 · 게인 3

바꾸는 것 · 방식: 한 번 보고 가기 → 계속 보며 따라가기 (`K` = 3)

목표	매 프레임 사진 속 **오차**를 보고 고치는 **시각 서보**가 움직이는 블록을 얼마나 잘 따라가는지 알아본다.
예상	**게인** $K_{\text{게인}}$ = 3 으로 매 프레임 **오차**를 고치면, 평균 추적 오차는 활동 1 의 약 51 mm 보다 얼마나 줄어들까? ☐ 1. 거의 그대로다 ☐ 2. 오히려 커진다 ☐ 3. 약 10 mm 로 크게 준다
할 일	`게인 K` 를 3 에 둔다. `계속 보며 따라가기` 를 누른다. 그다음 `이미지 자코비안` 을 켠다.
볼 것	카메라 가운데가 블록을 흔들림 없이 따라가 평균 추적 **오차**가 약 10 mm 로 활동 1 보다 훨씬 작고, 사진 속 빨간 오차 화살표가 3D 의 파란 손끝 화살표로 바뀐다
왜	1/30 초마다 사진 속 **오차** $e_{\text{사진 속 오차}}$ 를 다시 재서 그 반대쪽으로 움직이므로, 블록이 움직여 생긴 오차가 곧바로 고쳐진다. K = 3 은 오차 10 px 를 1 초에 30 px 빠르기로 줄이는 세기이고, $K \times \tau_{\text{지연}}$ = 3 × 0.2 = 0.6 이 한계 π/2 ≈ 1.57 보다 작다. 그래서 사진이 늦어도 조금 넘쳤다가 곧 가라앉는다. 다만 블록이 쉬지 않고 움직이므로 늘 조금 뒤처져 약 10 mm 의 오차는 남는다.
핵심	**계속 보며 고침 → 평균 오차 약 10 mm, 흔들림 없음**

3 활동 3 · 계속 보기 · 게인 9

바꾸는 것 · 게인 `K`: 3 → 9

목표	같은 **시각 서보**에서 **게인** K 만 크게 올리면 따라가는 모습이 어떻게 달라지는지 알아본다.
예상	K 를 3 에서 9 로 올리면 더 세게 고치니 **오차**가 더 줄어들까, 아니면 다른 일이 생길까? ☐ 1. 오차가 더 준다 ☐ 2. 흔들리며 오차가 커진다 ☐ 3. 활동 2와 똑같다
할 일	`게인 K` 만 9 로 올린다. 그다음 `계속 보며 따라가기` 를 누른다.
볼 것	K 가 한계(이 화면에서 약 7.2)를 넘어, 손끝이 블록을 지나쳤다 되돌아오기를 되풀이하며 흔들린다(흔들림 "큼", 평균 추적 **오차** 약 25 mm 로 활동 2 보다 커진다)
왜	사진은 0.2 초 늦게 들어오므로 로봇은 언제나 0.2 초 전의 **오차**를 보고 고친다. K = 9 로 세게 고치면 이미 줄어든 오차를 또 고쳐 블록을 지나치고, 지나친 것도 0.2 초 늦게 알아 반대로 또 지나친다. $K \times \tau$ = 9 × 0.2 = 1.8 이 한계 π/2 ≈ 1.57 을 넘고, K = 9 는 이 화면의 한계 약 7.2 도 넘었기 때문에, 고칠수록 흔들림이 커져 평균 추적 오차가 약 25 mm 로 늘어난다.
핵심	**K = 9 → 흔들리며 평균 오차 약 25 mm**

✎ 기록표: 화면의 결과를 적어요

활동	방식(한 번 보기 · 계속 보기 · 계속 보기)	게인 `K`(— · 3 · 9)	평균 추적 오차(mm)	흔들림(없음 · 없음 · 큼)
활동 1 한 번 보고				
활동 2 계속 보기 · 게인 3				
활동 3 계속 보기 · 게인 9				

정리: 세 활동 비교하기

세 번의 추적 오차 그래프를 겹쳐 놓는다. 보는 횟수와 게인에 따라 무엇이 달라지는지 본다. 평균 오차와 흔들림을 표로 본다.

알게 된 것:

정리 질문

게인 K 를 크게 할수록 항상 더 잘 따라갈까?

내 답:

기억할 것

보면서 움직이면(폐루프) 목표가 움직여도 따라간다. 다만 게인이 너무 크면 사진 지연 때문에 흔들린다.

현장에서는

컨베이어 위로 흘러오는 물건을 집는 로봇이 있다. 이 로봇은 카메라로 물건 위치를 한 번 잰다. 그 뒤에는 컨베이어 엔코더로 물건이 간 거리를 계속 계산해 따라가 집는다. 이것을 컨베이어 추적이라 한다. 용접 로봇도 카메라나 레이저 센서를 쓴다. 용접할 이음매를 보며 경로를 고친다. 그래서 부품이 조금씩 다른 자리에 놓여도 따라간다.

교사용 답

예상 정답: 활동 1: 1. 블록은 이미 다른 곳에 있다 · 활동 2: 3. 약 10 mm 로 크게 준다 · 활동 3: 2. 흔들리며 오차가 커진다

정리 질문: 아니다. 사진이 0.2 초 늦게 들어오므로 이 화면에서 K 가 약 7.2 를 넘으면 블록을 지나쳤다 되돌아오며 흔들려 오차가 커진다.

개념 확인 질문: 한 번 보고 이동하는 것과 시각 서보는 무엇이 다를까?

다음 차시 · 27차시 8-1 시연으로 배우기: 사람이 동작을 한 번 보여 주었다. 블록이 다른 자리에 있어도 따라 할 수 있을까?

시각 서보
visual servoing
카메라로 보면서 목표와의 차이를 줄이도록 계속 움직이는 제어.

피드백 제어
feedback (closed-loop) control
결과를 재서 목표와의 차이(오차)를 보고 명령을 계속 고치는 제어.

이미지 자코비안
Image Jacobian (interaction matrix)
카메라 속도와 사진 속 점의 속도를 잇는 표. $\dot{s} = L\,v$

한 번 보고 이동
Look-then-move
사진을 한 번 보고 위치를 계산한 뒤 보지 않고 움직이는 방식

게인
Gain
오차(또는 쌓인 오차·변화율)에 곱해 힘을 정하는 숫자. Kp·Ki·Kd

오차
error
목표와 지금 값의 차이.